In [1]:
```
print("Welcome to Data Analytics with Python!")
print("My name is Kazi")
print(25+80)
```

```
Welcome to Data Analytics with Python!
My name is Kazi
105
```

In [2]:
```
print("Total Sales for the Month:", 15000)
```

```
Total Sales for the Month: 15000
```

In [9]:
```
#This is a single-line comment
print("Hello Kazi")    #This prints a message

"""
This is a multi-line comment,
Used when you want to describe multiple steps.
"""
```

```
Hello Kazi
```

Out[9]:
```
'\nThis is a multi-line comment,\nUsed when you want to describe multiple s
teps.\n'
```

In [1]:
```
#Variable
name = "Islam"
age = 30
salary = 50000.40

print(name)
print(age)
print(salary)
```

```
Islam
30
50000.4
```

In [2]:
```
total_sales = 120000
total_customers = 400
average_sales = total_sales / total_customers
print(average_sales)
```

```
300.0
```

In [8]:
```
#Data Types
"""
int,
float, --25.5
str, --"Hello"
bool, --True/False
list, -- [1,2,3]
tuple, --(1,2,3)
dict --{"name":"Kazi", "age": 30}
"""
```

Out[8]: '\nint,\nfloat, --25.5\nstr, --"Hello"\nbool, --True/False\nlist, -- [1,2, 3]\ntuple, --(1,2,3)\ndict --{"name":"Kazi", "age": 30} \n'

In [10]:
```
x = 2
y = 3.5
name = "Kazi"
is_active = True
fruits = ["apple", "banana", "cherry"]
city = ("New York", "Albany", "Berlin")
person = {"name": "Monira", "age": 29}

print(type(x))
print(type(y))
print(type(name))
print(type(is_active))
print(type(fruits))
print(type(city))
print(type(person))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'list'>
<class 'tuple'>
<class 'dict'>
```

In [11]:
```
#Arithmetic Operations

a = 10
b = 3

print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a % b)
print(a ** b)
```

```
13
7
30
3.3333333333333335
1
1000
```

In [1]:
```
#Input from users
name = input("Enter your name: ")
age = input("Enter your age: ")

print("Hello", name, "You are", age, "years old.")
```

```
Hello Kazi You are 30 years old.
```

In [2]:
```
#To convert to an integer
age = int(input("Enter your age: "))
print("In 10 years, you will be ", age + 10)
```

```
In 10 years, you will be  40
```

In [ ]:

```

```

In [1]:
```
#Indentation
if 10 > 5:
    print("Ten is greater than five!")
```

```
Ten is greater than five!
```

In [2]:
```
sales = 12000
if sales > 10000:
    print("High sales day! Great job team")
```

```
High sales day! Great job team
```

In [3]:
```
a = 5
b = 10
sum = a + b
print(sum)
```

```
15
```

In [1]:
```
store_name = "Manhattan Mall"
total_sales = 130005.39
num_customer = 300
average_bill = total_sales / num_customer
print(average_bill)
```

```
433.3513
```

In [2]:
```
#Variable Assignment
a, b, c = 10, 20, 30
x = y = z = "Data"

print(a, b, c)
print(x, y, z)
```

```
10 20 30
Data Data Data
```

In [1]:
```
x = 10
x = "Now I am a string"

print(x)
```

```
Now I am a string
```

In [ ]:
```
#Python reserved words
if, for, while, class, return etc
```

In [2]:
```
import keyword
print(keyword.kwlist)
```

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break',
'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'fo
r', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'no
t', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

In [3]:
```
#Data types and Variables
a = 10
```

```
b = -25
c = 0

print(type(b))
```

```
<class 'int'>
```

In [1]:
```
price = 99.99
discount = 10.5
final_price = price - (price * discount / 100)
print("Final Price:", final_price)
```

```
Final Price: 89.49105
```

In [2]:
```
a = 5.5
b = 2.0
print(a + b)
print(a - b)
print(a * b)
print(a / b)
```

```
7.5
3.5
11.0
2.75
```

In [3]:
```
first_name = "Kazi"
last_name = "Islam"
full_name = first_name + " " + last_name
print(full_name)
```

```
Kazi Islam
```

In [4]:
```
print("python" * 3)
```

```
pythonpythonpython
```

In [5]:
```
text = "Data Analysis"
print(text[0:4])
print(text[5:13])
```

```
Data
Analysis
```

In [6]:
```
course = "python programming"
print(course.upper())
print(course.lower())
print(course.title())
print(course.replace("python", "data"))
print(len(course))
```

```
PYTHON PROGRAMMING
python programming
Python Programming
data programming
18
```

In [7]:
```
feedback = "    Great Product! very useful and affordable.  "
cleaned_feedback = feedback.strip().lower()
```

```
print(cleaned_feedback)
```

```
great product! very useful and affordable.
```

In [9]:
```
print(10 > 5)
print(10 < 5)
print(15 == 15)
print(20 != 10)
```

```
True
False
True
True
```

In [10]:
```
#Logical Operators
x = 10
print(x > 5 and x < 15)
print(x > 5 and x > 15)
```

```
True
False
```

In [11]:
```
x = 20
print(x < 25 and x == 20)
print(x > 15 and x < 10)
```

```
True
False
```

In [12]:
```
is_active = True
print(not is_active)
```

```
False
```

In [ ]:
```

```

In [1]:
```
#Conditional Statement
temperature = 35

if temperature > 30:
    print("It's a hot day! Stay hydrated.")
elif temperature >= 20:
    print("The weather is pleasant today.")
else:
    print("It's quite cold! Wear warm clothes.")
```

```
It's a hot day! Stay hydrated.
```

In [10]:
```
spending = 7500

if spending > 10000:
    category = 'Premium Customer'
elif spending >= 5000:
    category = 'Regular Customer'
else:
    category = 'Occasional Customer'
print(f"The customer is a {category}")
```

```
The customer is a Regular Customer
```

In [1]:
```
marks = 85
attendance = 50

if marks >= 80:
    if attendance >= 75:
        print("Grade A")
    else:
        print("Grade B (due to low attendance)")
else:
    print("Grade C")
```

```
Grade B (due to low attendance)
```

In [2]:
```
#Iterating over a list

fruits = ["apple", "banana", "mango"]
for fruit in fruits:
    print(f"I love {fruit}")
```

```
I love apple
I love banana
I love mango
```

In [3]:
```
for i in range(1, 6):
    print(f"Processing record {i}")
```

```
Processing record 1
Processing record 2
Processing record 3
Processing record 4
Processing record 5
```

In [4]:
```
count = 1
while count <= 5:
    print(f"Count is {count}")
    count +=1
```

```
Count is 1
Count is 2
Count is 3
Count is 4
Count is 5
```

In [5]:
```
#While loop in Data Cleaning

missing_values = 5
while missing_values > 0:
    print(f"Cleaning missing values ...Remaining: {missing_values}")
    missing_values -=1
```

```
Cleaning missing values ...Remaining: 5
Cleaning missing values ...Remaining: 4
Cleaning missing values ...Remaining: 3
Cleaning missing values ...Remaining: 2
Cleaning missing values ...Remaining: 1
```

In [6]:
```
for number in range(1, 6):
    if number == 3:
        continue #skip 3
    if number == 5:
        break #stop at 5
print(number)
```

```
5
```

In [3]:
```
#AND Operator
age = 25
experience = 1

if age > 21 and experience >= 2:
    print("Candidate is eligible for job.")
else:
    print("Candidate does not meet eligibility criteria")
```

```
Candidate does not meet eligibility criteria
```

In [5]:
```
#OR Operator
purchase_amount = 4000
is_premium_member = True

if purchase_amount > 5000 or is_premium_member:
    print("Customer is eligible for discount")
else:
    print("Customer is not eligible")
```

```
Customer is eligible for discount
```

In [1]:
```
#NOT Operator
```

```
is_premium_member = False

if not is_premium_member:
    print("Customer is not a premium member.")
else:
    print("Customer is premium")
```

```
Customer is not a premium member.
```

In [2]:
```
age = 28
experience = 4
degree = "Graduate"

if(age > 25 and experience > 3) or degree == "Postgraduate":
    print("Candidate is eligible for job.")
else:
    print("Candidate does not qualify.")
```

```
Candidate is eligible for job.
```

In [3]:
```
salary = 98000
credit_score = 750

if salary > 90000 or credit_score > 700:
    print("Loan approved.")
else:
    print("Loan rejected")
```

```
Loan approved.
```

In [6]:
```
#Checking the missing data by using not

customer_name = ""
#customer_name = "Kazi"

if not customer_name:
    print("Customer name missing! Please check your data. ")
else:
    print("Customer name available.")

#Output will be "Customer name available"
```

```
Customer name missing! Please check your data.
```

In [7]:
```
marks = 45
project_submitted = True
scholarship_student = True

if (marks > 50 and project_submitted) or scholarship_student:
    print("Student Passed")
else:
    print("Student Failed")
```

```
Student Passed
```

In [ ]:

In [ ]:
```
#Function block

def function_name(parameters):
    #code block
    return result
```

In [3]:
```
def greet():
    print("Hello, welcome to the Python World!")

greet()
```

```
Hello, welcome to the Python World!
```

In [4]:
```
a = 3
b = 4

def add_numbers(a, b):
    result = a + b
    return result

sum_result = add_numbers(10, 20)
print("The sum is: ", sum_result)
```

```
The sum is:  30
```

In [ ]:
```
#Use of function in Data Analytics such as data cleaning, remove redundent c

def clean_data(df):
    df = df.dropna()
    df = df.drop_duplicates()
    return df
```

In [6]:
```
#Creating and Importing a Module
a = 5
b = 8

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        return "Cannot be divided by zero"
    else:
        return a / b

print("Multiplication result:", multiply(a, b))
print("Division result:", divide(a, b))
```

```
Multiplication result: 40
Division result: 0.625
```

In [7]:
```
#Global Variable
company_name = "Data Insights pvt Ltd"

def show_company ():
```

```
    print("Company name is: ", company_name)

#Calling the function
show_company()

#Accessing outside the function
print("We work at", company_name)
```

```
Company name is:  Data Insights pvt Ltd
We work at Data Insights pvt Ltd
```

In [8]:
```
def student_info():
    name = "Kazi"
    grade = "A"
    print("Inside Function:", name, grade)

student_info()

#Trying to access outside
print("outside function:", name)
```

```
Inside Function: Kazi A
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[8], line 9
      6 student_info()
      8 #Trying to access outside
----> 9 print("outside function:", name)

NameError: name 'name' is not defined
```

In [9]:
```
count = 0 #Global variable

def increase_count():
    global count  #Accessing the global variable
    count += 1
    print("Count inside function:", count)

increase_count()
increase_count()
increase_count()

print("Count outside function:", count)
```

```
Count inside function: 1
Count inside function: 2
Count inside function: 3
Count outside function: 3
```

In [10]:
```
def demo_lifetime():
    x = 5  #Local variable created
    print("x inside function:", x)

demo_lifetime()
print("Trying to access x outside:", x)
```

```
x inside function: 5
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[10], line 6
      3     print("x inside function:", x)
      5 demo_lifetime()
----> 6 print("Trying to access x outside:", x)

NameError: name 'x' is not defined
```

In [3]:
```
a = 100  #Global variable
def test_scope():
    b = 50 #Local variable
    print("Inside function:")
    print("a =", a)
    print("b =", b)

test_scope()

print("\nOutside function:")
print("a =", a)
print('Trying to print b will cause an error below: ')
print("b =", b)
```

```
Inside function:
a = 100
b = 50

Outside function:
a = 100
Trying to print b will cause an error below:
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[3], line 13
     11 print("a =", a)
     12 print('Trying to print b will cause an error below: ')
---> 13 print("b =", b)

NameError: name 'b' is not defined
```

In [4]:
```
import math
print("Square root of 25:", math.sqrt(25))
```

```
Square root of 25: 5.0
```

In [5]:
```
print("value of pi: ", math.pi)
```

```
value of pi:  3.141592653589793
```

In [6]:
```
import pandas as pd
data = {"Name": ["Kazi", "Islam", "Monira"], "Score": [88, 92, 79]}
df = pd.DataFrame(data)
print(df)
```

```
     Name  Score
0    Kazi     88
1   Islam     92
2  Monira     79
```

In [7]:
```
import random
print("Random integer between 1 and 10:",random.randint(1, 10))
print("Random floating-point number between 0 and 1:", random.random())
```

```
Random integer between 1 and 10: 1
Random floating-point number between 0 and 1: 0.12174981244215966
```

In [9]:
```
import os
print("Current working Directory:", os.getcwd())

print("Files in current directory:", os.listdir())
```

```
Current working Directory: /private/var/mobile/Containers/Data/Application/0
6DFBEBA-74FA-48CE-8D41-EF809EE54699/Documents
Files in current directory: ['PythonForDataAnalytics2.html', 'PythonForDataA
nalytics3.asciidoc', 'PythonForDataAnalytics2.asciidoc', '.config', 'PythonF
orDataAnalytics3.ipynb', 'PythonForDataAnalytics.html', 'PythonForDataAnalyt
ics3.html', 'PythonForDataAnalytics2.ipynb', 'PythonForDataAnalytics4.ipyn
b', '.ipython', '.ipynb_checkpoints', '.jupyter', 'PythonForDataAnalytics.ip
ynb', '.Trash', 'PythonForDataAnalytics.asciidoc']
```

In [10]:
```
import time

current_time = time.ctime()
print("Current time is:", current_time)
```

```
Current time is: Sun Apr 12 03:50:34 2026
```

In [ ]:

In [2]:
```
import pandas as pd

#creating a series from a list
data = [10, 20, 30, 40]
series = pd.Series(data)

print(series)

series = pd.Series([10, 20, 30, 40], index = ['a', 'b', 'c', 'd'])
print(series)
```

```
0    10
1    20
2    30
3    40
dtype: int64
a    10
b    20
c    30
d    40
dtype: int64
```

In [6]:
```
data = {
    'Name': ['Kazi', 'Monira', 'Bushra'],
    'Age' : [23, 20, 10],
    'City': ['NY', 'London', 'Frankfurt']

}

df = pd.DataFrame(data)
print(df)
```

```
     Name  Age       City
0    Kazi   23         NY
1  Monira   20     London
2  Bushra   10  Frankfurt
```

In [7]:
```
#File Types
pd.read_csv() - For csv files
pd.read_excel() - For excel files
pd.read_json() - For json data
pd.read.sql() - For sql databases
pd.read_html - For tables extracted directly From webpages
```

```
  Cell In[7], line 2
    pd.read_csv() - For csv files
                  ^
SyntaxError: invalid syntax
```

In [ ]:
```
#CSV file
import pandas as pd
df = pd.read_csv("sales_data.csv", delimeter = ";", encoding = "utf-8", na_v
print(df)

#Importing excel files
```

```
df = pd.read_excel("employee_data.xlsx", sheet_name = "sheet1")

#Importing Json Data
df = pd.read_json("customer_data.json")


to_csv() - export to csv
to_excel() - export to excel
to_json() - export to json
to_sql() - export to sql databases
```

In [2]:
```
import pandas as pd

data = {
    'Name': ['Kazi', 'Monira', 'Bushra'],
    'Age' : [23, 20, 10],
    'City': ['NY', 'London', 'Frankfurt'],
    "Department" : ['HR', 'Finance', 'IT']

}

df = pd.DataFrame(data)

#Select one column
print(df["Name"])

#select multiple columns
print(df[["Name", "Age"]])
```

```
0      Kazi
1    Monira
2    Bushra
Name: Name, dtype: object
     Name  Age
0    Kazi   23
1  Monira   20
2  Bushra   10
```

In [3]:
```
#Selecting rows
print(df[0:2])
```

```
     Name  Age    City Department
0    Kazi   23      NY         HR
1  Monira   20  London    Finance
```

In [4]:
```
#selecting any specific value
print(df["Age"][1])
```

```
20
```

In [6]:
```
#label based indexing
print(df.loc[[0, 2], ["Name", "Department"]])
```

```
     Name Department
0    Kazi         HR
2  Bushra         IT
```

In [7]:
```
#integer location based indexing
print(df.iloc[0])
```

```
Name          Kazi
Age             23
City            NY
Department      HR
Name: 0, dtype: object
```

In [9]:
```
#Employee older than 20
print(df[df["Age"] > 20])
```

```
   Name  Age City Department
0  Kazi   23   NY         HR
```

In [12]:
```
print(df[(df["Age"] > 9) & (df["Department"] == "IT")])
```

```
     Name  Age       City Department
2  Bushra   10  Frankfurt         IT
```

In [14]:
```
#identifying

import pandas as pd
import numpy as np

data = {
    'Name': ['Kazi', 'Monira', 'Bushra', 'Noor', 'Shirajum'],
    'Age' : [23, np.nan, 20, 10, np.nan],
    'City': ['NY', 'London', 'Frankfurt', np.nan, 'Berlin']
}

df = pd.DataFrame(data)
print("\nDataFrame:")
print(df)

print("\nCount of missing values: ")
print(df.isnull())

print(df.isnull().sum())
```

```
DataFrame:
       Name   Age       City
0      Kazi  23.0         NY
1    Monira   NaN     London
2    Bushra  20.0  Frankfurt
3      Noor  10.0        NaN
4  Shirajum   NaN     Berlin

Count of missing values:
    Name    Age   City
0  False  False  False
1  False   True  False
2  False  False  False
3  False  False   True
4  False   True  False
Name    0
Age     2
City    1
dtype: int64
```

In [ ]:

In [2]:
```
#project Student marks analysis

#import necessary libraries
import pandas as pd
import  matplotlib.pyplot as plt

#create a simple tabular dataset
data = {
    'Student': ['Kazi', 'Islam', 'Monira', 'Bushra', 'Noor'],
    'Math': [60, 88, 69, 59, 90],
    'Science': [58, 79, 69, 96, 80],
    'English': [77, 87, 57, 74, 99]
}

#Create dataframe
df = pd.DataFrame(data)

#Display table
df
```

Out[2]:

	Student	Math	Science	English
0	Kazi	60	58	77
1	Islam	88	79	87
2	Monira	69	69	57
3	Bushra	59	96	74
4	Noor	90	80	99

In [3]:
```
#calculate average marks
#Calculate average marks for each student
df['Average'] = df[['Math', 'Science', 'English']].mean(axis = 1)

#Display updated DataFrame
df
```

Out[3]:

	Student	Math	Science	English	Average
0	Kazi	60	58	77	65.000000
1	Islam	88	79	87	84.666667
2	Monira	69	69	57	65.000000
3	Bushra	59	96	74	76.333333
4	Noor	90	80	99	89.666667

In [4]:
```
#Assign grades
#Function to assign grades
def assign_grade(avg):
    if avg >= 90:
```

```
        return 'A'
    elif avg >= 80:
        return 'B'
    elif avg >= 70:
        return 'C'
    elif avg >= 60:
        return 'D'
    else:
        return 'F'

#Apply the function to calculate grades
df['Grade'] = df['Average'].apply(assign_grade)

#Display DataFrame with grades
df
```

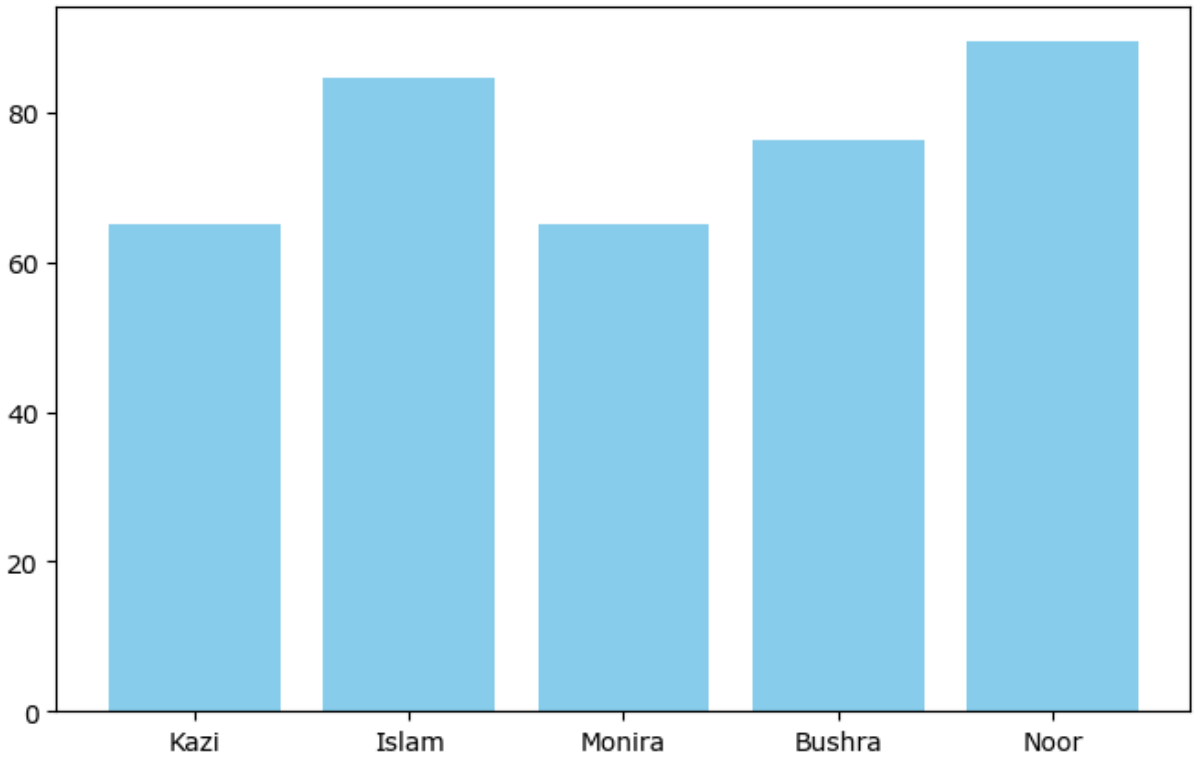
Out[4]:

	Student	Math	Science	English	Average	Grade
0	Kazi	60	58	77	65.000000	D
1	Islam	88	79	87	84.666667	B
2	Monira	69	69	57	65.000000	D
3	Bushra	59	96	74	76.333333	C
4	Noor	90	80	99	89.666667	B

In [5]:

```
#Basic visualization
#Plot average marks of students

plt.figure(figsize=(8,5))
plt.bar(df['Student'], df['Average'], color = 'skyblue')
plt.xlable('Students')
plt.ytable('Average Marks')
plt.show()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[5], line 6
      4 plt.figure(figsize=(8,5))
      5 plt.bar(df['Student'], df['Average'], color = 'skyblue')
----> 6 plt.xlable('Students')
      7 plt.ytable('Average Marks')
      8 plt.show()

AttributeError: module 'matplotlib.pyplot' has no attribute 'xlable'
```



In [14]:
```
#Titanic Data Analysis
#Analysis the Titanic dataset to understand passenger demographics, survival

#import libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

#optional: improve plot style
#sns.set_style('whitegrid')
plt.style.use('seaborn-v0_8-whitegrid')

#load dataset from a csv url
#url = "https://www.kaggle.com/datasets/yasserh/titanic-dataset"
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/tit
df = pd.read_csv(url)

#display first 5 rows
df.head()
```

Out[14]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	7
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	5
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	

In [6]:

```
#shape of the dataset (rows, columns)
print("Dataset shape:", df.shape)

#Basic info
df.info()

#Summary statistics for numeric columns
df.describe()
```

```
Dataset shape: (5, 6)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 6 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   Student  5 non-null      object
 1   Math     5 non-null      int64
 2   Science  5 non-null      int64
 3   English  5 non-null      int64
 4   Average  5 non-null      float64
 5   Grade    5 non-null      object
dtypes: float64(1), int64(3), object(2)
memory usage: 372.0+ bytes
```

Out[6]:

	Math	Science	English	Average
count	5.000000	5.000000	5.0000	5.000000
mean	73.200000	76.400000	78.8000	76.133333
std	14.956604	14.117365	15.6269	11.223982
min	59.000000	58.000000	57.0000	65.000000
25%	60.000000	69.000000	74.0000	65.000000
50%	69.000000	79.000000	77.0000	76.333333
75%	88.000000	80.000000	87.0000	84.666667
max	90.000000	96.000000	99.0000	89.666667

In [15]:

```
#Handle the missing data
#Check missing values

df.isnull().sum()
```

Out[15]:

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

In [19]:

```
#Fill the missing values with median
df['Age'].fillna(df['Age'].median(), inplace = True)
#df['Age'] = df['Age'].fillna(df['Age'].median())

#Drop rows where embarked is missing
df.dropna(subset = ['Embarked'], inplace = True)

#Optional: Drop Cabin column due to too many missing values
df.drop(columns = ['Cabin'], inplace = True, errors = 'ignore')

# always check columns first
#print(df.columns)

# safe dropping
#df.drop(columns=['Cabin'], inplace=True, errors='ignore')
```

```
/private/var/mobile/Containers/Data/Application/06DFBEBA-74FA-48CE-8D41-EF80
9EE54699/tmp/ipykernel_3213/2354674791.py:2: FutureWarning: A value is tryin
g to be set on a copy of a DataFrame or Series through chained assignment us
ing an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behave
s as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'd
f.method({col: value}, inplace=True)' or df[col] = df[col].method(value) ins
tead, to perform the operation inplace on the original object.


  df['Age'].fillna(df['Age'].median(), inplace = True)
```

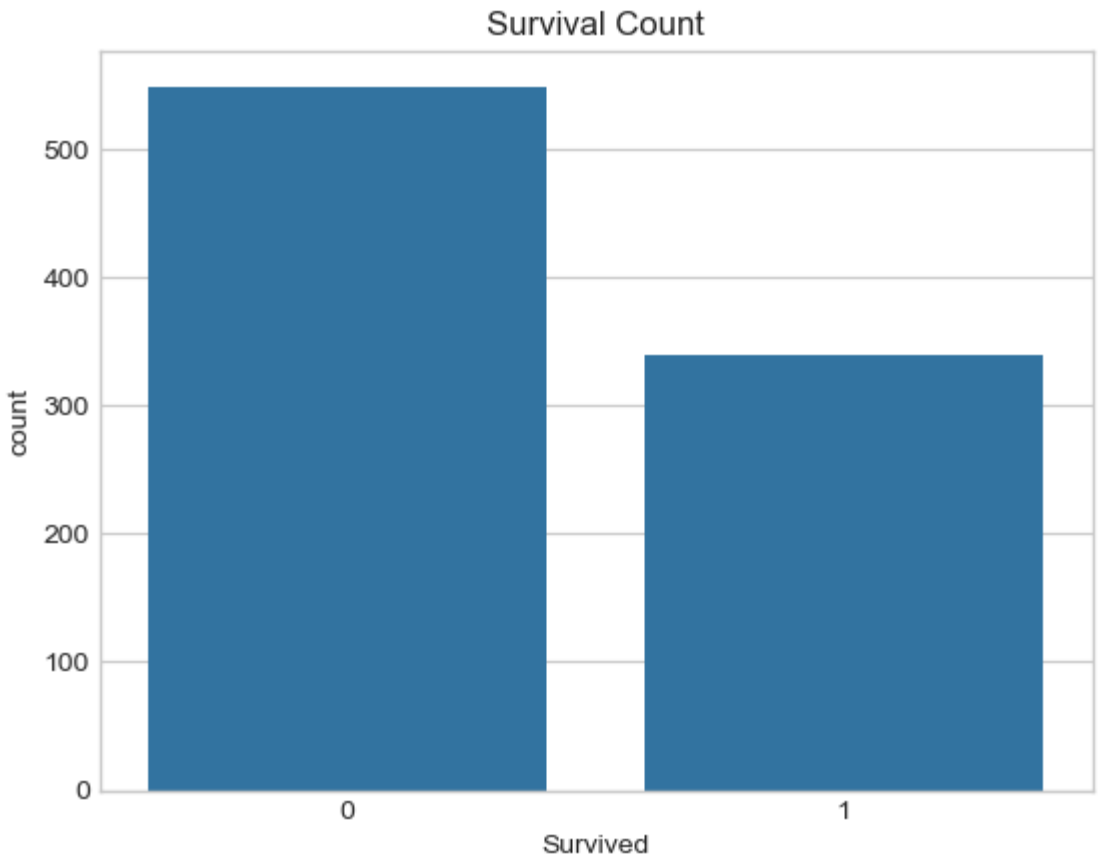
In [20]:
```
#Check missing values
df.isnull().sum()

#Fill missing age values
df['Age'] = df['Age'].fillna(df['Age'].median())

#Drop rows where Embarked is missing
df = df.dropna(subset = ['Embarked'])
```

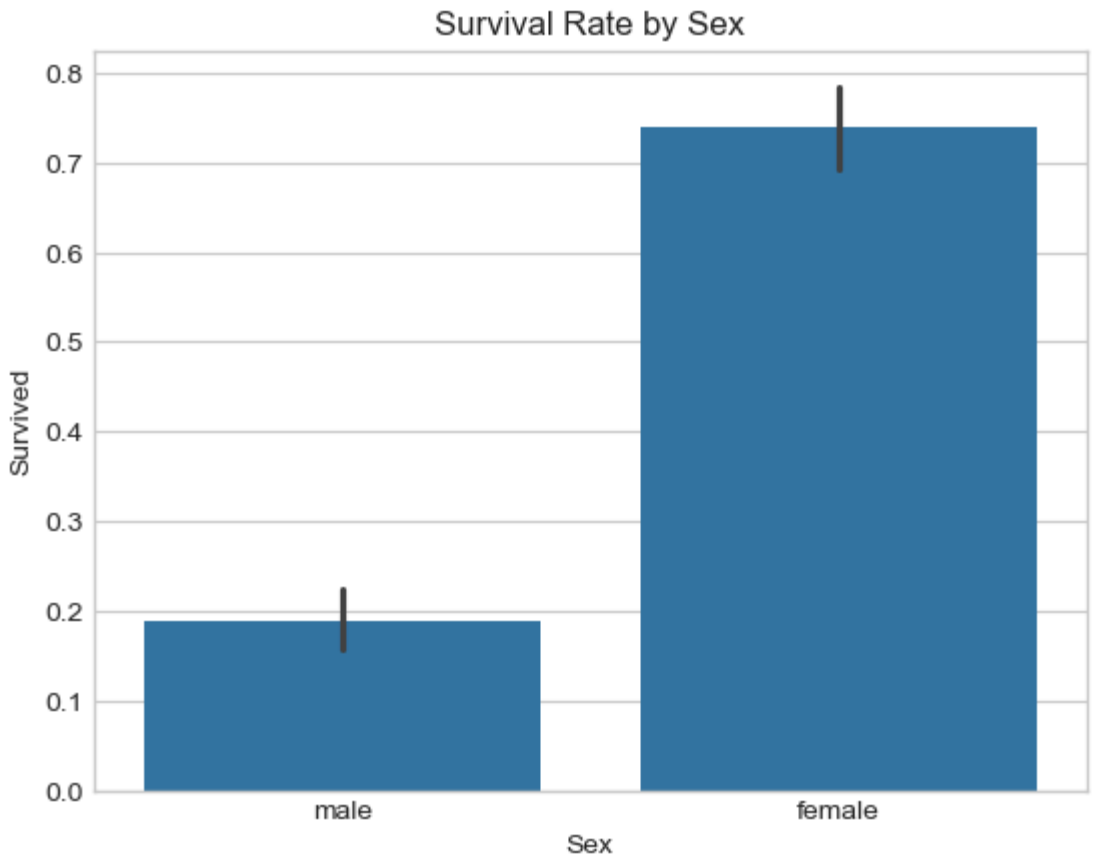
In [21]:
```
#Basic analysis
#How many passengers survived?
survival_count = df['Survived'].value_counts()
print(survival_count)

#Survival rate by sex
print(df.groupby('Sex')['Survived'].mean())

#Survival rate by passenger class
print(df.groupby('Pclass')['Survived'].mean())
```

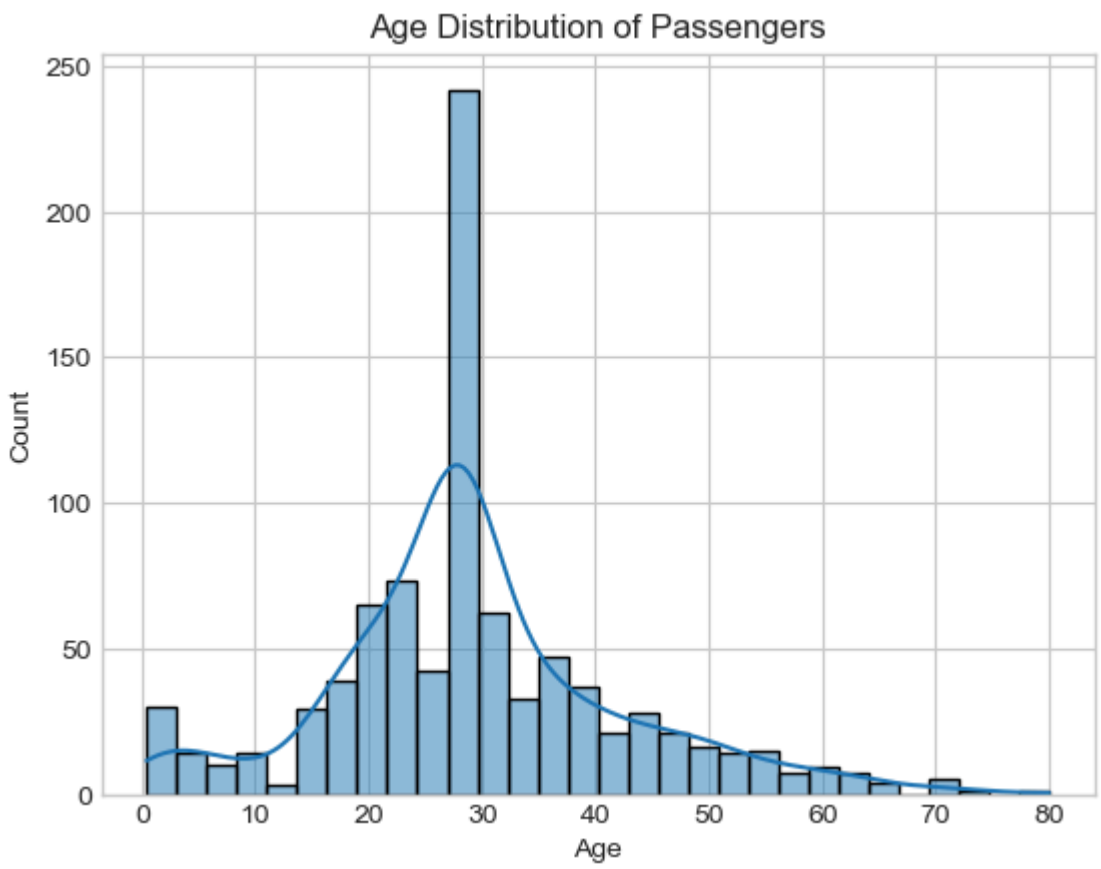
```
Survived
0    549
1    340
Name: count, dtype: int64
Sex
female    0.740385
male      0.188908
Name: Survived, dtype: float64
Pclass
1    0.626168
2    0.472826
3    0.242363
Name: Survived, dtype: float64
```

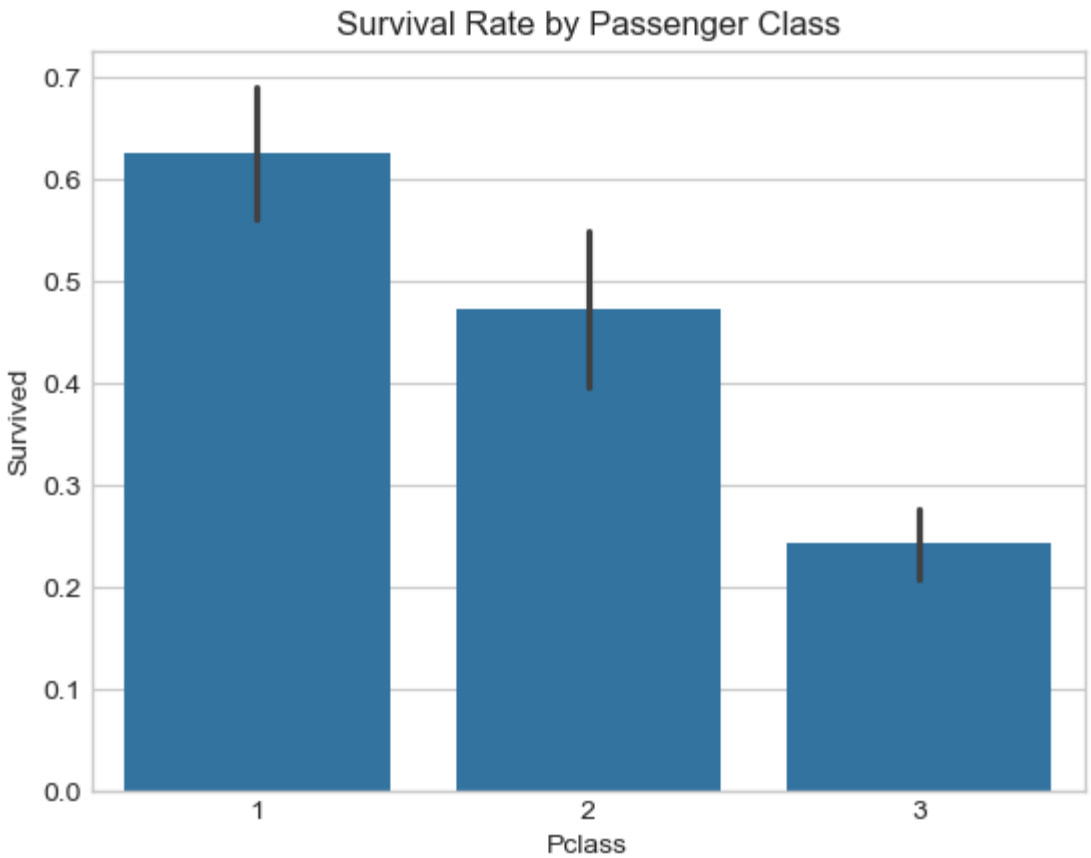
In [22]:
```
#Basic Visualization

#Plot survival counts
sns.countplot(x = 'Survived', data = df)
plt.title('Survival Count')
plt.show()
```

```
#Survival by sex
sns.barplot(x = 'Sex', y = 'Survived', data = df)
plt.title('Survival Rate by Sex')
plt.show()


#Age distribution
sns.histplot(df['Age'], bins = 30, kde = True)
plt.title('Age Distribution of Passengers')
plt.show()

#Survival by Pclass
sns.barplot(x = 'Pclass', y = 'Survived', data = df)
plt.title('Survival Rate by Passenger Class')
plt.show()
```

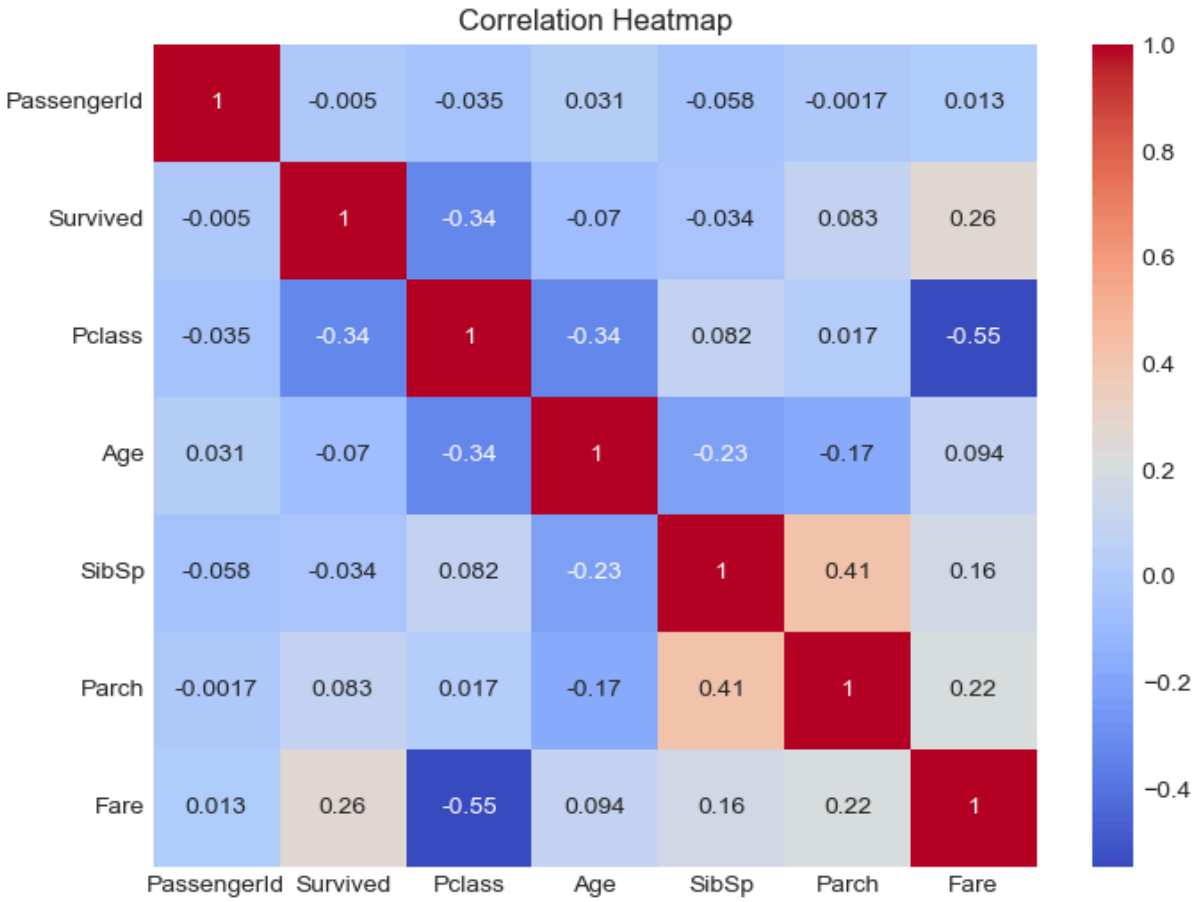



Survival Rate by Sex
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0
Survived
male
female
Sex


Age Distribution of Passengers
250
200
150
100
50
0
Count
0
10
20
30
40
50
60
70
80
Age



In [24]:
```
#Correlation Analysis

#Correlation between numeric columns

# Create correlation matrix
corr = df.corr(numeric_only=True)
#Heatmap
plt.figure(figsize =(8, 6))
sns.heatmap(corr, annot = True, cmap = 'coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```


Correlation Heatmap
PassengerId
Survived
Pclass
Age
SibSp
Parch
Fare
1 -0.005 -0.035 0.031 -0.058 -0.0017 0.013
-0.005 1 -0.34 -0.07 -0.034 0.083 0.26
-0.035 -0.34 1 -0.34 0.082 0.017 -0.55
0.031 -0.07 -0.34 1 -0.23 -0.17 0.094
-0.058 -0.034 0.082 -0.23 1 0.41 0.16
-0.0017 0.083 0.017 -0.17 0.41 1 0.22
0.013 0.26 -0.55 0.094 0.16 0.22 1
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4

In [ ]: